

Owo Reach

Payroll for the informal economy.

One list in, everyone paid, bank account or not, with every naira accounted for.

Team document, v1

Monnify Developer Challenge, API Conference Lagos 2026

Submission deadline: 12pm WAT, Monday July 21, 2026

1. Executive summary

Owo Reach is a payout platform for organisations that need to pay people who may or may not have bank accounts. The admin uploads one list of beneficiaries, messy or clean. People with accounts are verified by name enquiry and paid by instant bank transfer. People without accounts receive a Monnify Paycode by SMS, a 10-digit code they exchange for cash at any Moniepoint agent. Every payment is then tracked through its full life on a live dashboard, so the organisation always knows the answer to one question: did everyone get paid?

We are building this for the Monnify Developer Challenge because it rests on the one capability that is exclusive to Monnify (Paycode), it addresses a real and chronic problem in Nigeria (paying the unbanked safely and auditably), and it can be executed to a high standard by two people in six days if we hold scope to a single golden path.

This document is the single source of truth for the team. It contains the problem case, the product definition, the technical design, the Monnify integration details, the day-by-day plan, the risks with their fallbacks, and the demo script. If reality contradicts anything here, we update the document, not our memory.

2. The problem

A large share of Nigerian adults are unbanked or underbanked. Many have no smartphone. But organisations still need to pay them, every single month. A cooperative settling farm-gate prices with smallholders. An NGO distributing stipends or cash aid. A church running a welfare list. A construction firm paying day labourers. A market association returning contributions.

Today these organisations solve it the only way available to them: someone withdraws a brick of cash and physically shares it out. That method fails on every axis that matters:

- **Unsafe.** A person carrying payroll cash through Lagos or rural Kaduna is a target.
- **Unauditable.** A signature on a paper list proves nothing to a donor, a board, or an accountant.
- **Leaky.** Weeks later, nobody can say with certainty who received what.
- **Slow.** A payout that should take minutes consumes days of staff time.

Existing digital tools do not fix this, because every payroll product and every payout API integration begins with the same field: enter the recipient's account number. For millions of the intended recipients, that field cannot be filled. So the problem, stated in one line: organisational money stops where bank accounts stop, and the people past that line are the ones who most need reliable payment.

3. The solution

Owo Reach removes the assumption instead of working around it. One list goes in, and every person on it is paid through whichever rail actually reaches them.

The sender's experience

An org admin creates a payout run by uploading a spreadsheet, pasting text from WhatsApp, or typing free-form. The AI layer normalises whatever arrives into a clean beneficiary list with names, phone numbers, amounts, and bank details where they exist. It flags duplicates, outlier amounts, and missing data. For every beneficiary with an account number, Monnify Name Enquiry checks that the account resolves and that the name on the bank record matches the name on the list. Mismatches are flagged for review, never silently paid. The admin sees one review screen with totals, fees, and flags, plus a short plain-language brief written by the AI. One click approves the run.

The recipient's experience

A recipient with a bank account simply receives money, instantly. A recipient without one receives an SMS: you have received NGN 20,000 from Green Harvest Co-op, show this code with your ID at any Moniepoint agent to collect cash, expires in 72 hours. She needs no app, no smartphone, no signup, no KYC ceremony. She needs the phone she already owns and a walk to an agent she already knows. Moniepoint's agent network numbers in the hundreds of thousands and reaches far past where bank branches stop.

The part nobody builds: after the send

Every transfer and every paycode lives in a tracked lifecycle: queued, sent or code issued, completed, failed, expiring soon, expired, reissued, cancelled. State changes arrive by webhook and appear on the dashboard in real time. Codes about to expire trigger an automatic SMS nudge. Expired codes can be cancelled and reissued in one click, returning funds to the run. If a recipient loses the SMS, the admin can reveal the code again through an explicit authorized action that is written to the audit log. This lifecycle management is the difference between a demo and a product, and it is where most of our engineering effort goes.

4. Scope: the golden path and nothing else

Six days and two people means one workflow, executed beautifully. Everything below in the first list ships. Everything in the second list does not, no matter how good it sounds on day three.

In scope

- Create a payout run from a messy list (CSV, paste, or free text) with AI normalisation and flagging.
- Name enquiry verification for banked beneficiaries, with loose name matching suited to Nigerian bank records.
- One review screen: totals, fees (flat NGN 100 per paycode), flags, AI pre-flight brief, one approve button.
- Execution: bank transfers via the Disbursement API (including the sandbox OTP step), paycodes via the Paycode API, and a composed SMS per unbanked recipient (sender is stubbed; the message body is shown honestly in the UI).
- Live dashboard: per-beneficiary states driven by webhooks, streamed to the browser over SSE.
- Lifecycle actions: expiry nudge, one-click reissue, cancel, and audited clear-code reveal.

Out of scope, deliberately

- Recipient-facing app of any kind.
- Multi-organisation auth, roles, or tenancy.
- Wallets, bills payment, airtime, or any second product.
- Real SMS gateway integration (we stub it and say so).
- Anything mobile-native.

5. Where AI lives, and where it does not

The challenge brief encourages AI but warns that AI slop is frowned upon. Our rule: AI earns its place only where the work is genuinely hard without a model, and every AI feature must protect money before it moves. Two features qualify.

- **Chaos ingestion.** Beneficiary lists arrive as WhatsApp pastes, ragged spreadsheets, and photos of handwritten pages. The model extracts name, phone, amount, and bank details from whatever arrives, and flags duplicates, outliers, and gaps. Output streams into the review screen token by token so there is never a dead spinner.
- **Pre-flight brief.** Before approval, a short plain-language summary: 23 recipients, NGN 415,000 total, two name mismatches, one duplicate phone, estimated fees NGN 2,300. AI as a safety layer, not a gimmick.

Banned: chatbots, AI-generated imagery, and ask-AI-anything panels. In a field where many entries will bolt a chat window onto a checkout page, restraint is the differentiator.

6. Architecture and stack

Single repo, single deployable. The backend is Bun with Hono, Drizzle ORM, and SQLite. The frontend is Vite, React, TanStack Query, and Tailwind. In production the Hono process also serves the built frontend, so there is one process, one public URL, and that same URL is the webhook endpoint. No CORS, no frontend and backend drifting apart on demo day.

Why these choices

- **Bun + Hono:** near-instant startup, one language across API, scripts, and frontend, and one of the fastest JS web frameworks with built-in SSE helpers.
- **SQLite + Drizzle:** judges clone the repo and run four commands with no Docker and no database server. The README notes that swapping to Postgres is a one-line driver change.
- **SSE over WebSockets:** dashboard traffic is strictly server to client. EventSource reconnects automatically and the whole thing is twenty lines instead of a socket server.
- **Vite SPA over Next.js:** this is an authenticated dashboard, not a content site. No SSR or SEO needs, and the dev loop is faster.

Non-negotiable engineering rules

- Money is integer kobo everywhere. Floats exist only at the Monnify API boundary, which takes naira decimals.
- Webhook-driven, never polled. Accept, persist the event, return 200 fast, process async, push over SSE. The status API is a reconciliation fallback only.
- Every state change goes through one transition function that persists, audit-logs, and broadcasts. No side doors.
- The events table is append-only. It is the audit trail and the dashboard activity feed at the same time.
- Optimistic UI. Approval moves the instant the button is pressed and reconciles when the API confirms.

Repository layout

```
web/ (Vite + React UI) | server/monnify/ (API client: cached token, transfers with
OTP, paycodes, verification, webhook HMAC) | server/routes/ (runs, webhooks, SSE
events) | server/services/ (payout state machine) | server/db/ (schema, client,
seed) | scripts/day1-spike.ts (sandbox smoke test) | docs/ (this document, PRD,
rationale)
```

7. Monnify integration map

Sandbox base URL: <https://sandbox.monnify.com>. Live APIs are prohibited by the rules. Every endpoint path is defined once in `server/monnify/config.ts`; if a path is wrong, it is fixed in that one file.

Capability	Endpoint	Used for
Auth	POST <code>/api/v1/auth/login</code>	Bearer token, cached until 60s before expiry
Banks list	GET <code>/api/v1/banks</code>	Bank picker and code lookup

Capability	Endpoint	Used for
Name enquiry	GET /api/v1/disbursements/account/validate	Verify banked beneficiaries before paying
Single transfer	POST /api/v2/disbursements/single	Banked payouts
Authorize transfer	POST .../single/validate-otp (verify path)	Sandbox MFA returns PENDING_AUTHORIZATION
Transfer status	GET .../single/summary?reference=	Reconciliation fallback for missed webhooks
Paycode create	POST /api/v1/paycode (verify path)	Unbanked payouts, flat NGN 100 fee
Paycode get / clear / cancel / list	See config.ts, verify in spike	Lifecycle: masked view, audited reveal, reissue
Webhooks	HMAC-SHA512 of raw body, monnify-signature header	All state transitions

Sandbox facts that will bite us if forgotten

- MFA is on by default for disbursements. Transfers return PENDING_AUTHORIZATION and need an OTP step. We surface this in the UI as a maker-checker approval, which is honestly a better product anyway.
- Paycode redemption is simulated from the dashboard: Developer, then Simulator tab. This is how the demo video shows the full loop.
- The webhook URL is set in dashboard settings and needs a public URL. Cloudflare Tunnel in dev; the production URL once deployed.
- Paycode endpoint paths carry verify notes. The day-1 spike (bun run spike) exercises all six risky integration points and reports exactly what needs correcting.

8. Data model

Table	Purpose	Key fields
payout_runs	One approved batch	id, title, status (DRAFT / REVIEW / EXECUTING / COMPLETED / PARTIAL / FAILED), totals in kobo, AI pre-flight brief
beneficiaries	One person to pay	name, phone, amountKobo, rail (BANK / PAYCODE), account details, name enquiry result and match, status, monnifyReference, flags, smsBody
events	Append-only log	runId, beneficiaryId, type, payload, createdAt. Doubles as audit trail and activity feed

Beneficiary states: PENDING_REVIEW, QUEUED, SENT (transfer initiated), CODE_ISSUED (paycode created and SMS composed), COMPLETED (transfer confirmed or code redeemed), FAILED, EXPIRED, CANCELLED. Our Monnify references are always owo- followed by the beneficiary id, so webhooks map back to a row with no lookup table.

9. Security posture

- Webhook signatures are validated with HMAC-SHA512 over the raw body bytes, compared with a timing-safe check, before a single byte is trusted.
- Paycodes are stored and displayed masked. Revealing a clear code is an explicit, authorized action written to the audit log, because a live cash code is sensitive.
- Sandbox keys only, loaded from .env, with .env.example committed and the real file ignored. Before the repo goes public we audit history for secrets.
- The judging criteria call out exposed secrets and errors specifically. Treat both as disqualifying bugs, not polish items.

10. Six-day plan and division of labour

Day	Person A (backend, integration)	Person B (frontend, product)	Together
Wed 15	Run day-1 spike end to end; verify paycode paths; get a real webhook landing via tunnel	Repo and Vite shell, design tokens, dashboard skeleton	Slack intros, sandbox keys, conference tickets, social post with both hashtags
Thu 16	Harden Monnify client: token cache, OTP flow, webhook validation; runs and execute service	Run creation flow and review screen	Agree the API contract
Fri 17	Paycode lifecycle: issue, cancel, reissue, audited reveal; SSE events	Live dashboard, beneficiary states, activity feed	
Sat 18	AI ingestion and pre-flight brief, streamed	Polish golden path, empty and error states	Integration test the full flow
Sun 19	Seed data; reconciliation fallback job	README walkthrough; record demo video	Feature freeze at noon
Mon 20	Bug bash; secrets audit; fresh-clone setup test	Video edit; final README pass	Submit in Slack. Do not wait for the 21st
Tue 21	Buffer only	Buffer only	Deadline 12pm WAT

11. Risks and fallbacks

Risk	Trigger	Response
Paycode weak or broken in sandbox	Day-1 spike still failing after 3 hours of effort	Pivot the unbanked rail to SMS plus reserved accounts. The product story survives intact
OTP flow blocks automation	OTP not retrievable programmatically	Surface OTP entry in the admin UI as a maker-checker step and present it as a feature, because it is one
Flaky webhook delivery	Missed events during testing	Reconciliation job polls the status API for stale in-flight items every 60 seconds
Scope creep	Anyone says: wouldn't it be cool if	The answer is no. Golden path only

12. Demo video script (2 to 5 minutes)

- Cold open on the problem, not the product: every month this cooperative pays 200 farmers, and forty of them have no bank account.
- The messy list goes in. The AI cleans it on screen and catches a duplicate and a name mismatch.
- One click to approve. Split screen: transfers fire, paycodes issue.
- The recipient's phone shows the SMS with the code. Cut to the Simulator: the agent redeems it.

- The dashboard flips to COMPLETED live. Close on the reconciliation view: every naira accounted for.

One story, no feature tours. Rehearse twice, record once. The video is a scored deliverable, not an afterthought.

13. How this maps to the judging criteria

Criterion	How Owo Reach answers it
Solves a real problem / useful Monnify workflow	A documented, chronic problem (paying the unbanked safely and auditably) built on the one workflow exclusive to Monnify
Clarity and storytelling	The entire demo is one person's story. Amina has no bank account and gets paid in ninety seconds
Technical depth	Webhook-driven state machine, HMAC-validated events, full paycode lifecycle, sandbox MFA handled as maker-checker, integer-kobo money handling
No errors, broken links, or exposed secrets	SQLite means a four-command cold start; secrets audited; honest labelling of what is stubbed
Public repo with step-by-step local setup	Setup tested on a clean machine before submission; the spike script lets judges verify their own sandbox in minutes

The hard questions, answered in advance

Isn't this just a wrapper on Monnify's APIs?

Every fintech is a wrapper on rails. The product is the workflow around them: ingestion, verification, lifecycle tracking, reconciliation. That is the gap between an API and a solution, which is exactly what the challenge asks us to close.

The SMS is stubbed and it only runs in sandbox.

The rules require sandbox. We state plainly, in the UI and the README, what is simulated. Honest seams beat hidden ones.

Would anyone actually pay for this?

Organisations already pay for this problem today, in staff days, transport, missing cash, and failed audits. A flat NGN 100 per cash payout with a complete audit trail costs far less than the envelope it replaces.

14. Submission checklist

- Working prototype deployed (Railway or Fly), sandbox keys only.
- Public repo, .env.example present, zero secrets anywhere in git history.
- README: problem, demo GIF, architecture, cold-start setup steps tested on a clean machine.
- Demo video, 2 to 5 minutes, linked.
- Both of us registered attendees with conference tickets (in-person attendance is required to win).

- Introduced in apiconf-hackathon with notifications on; submission link followed from there.
 - Social post with #APIConfXMonnify and #DeveloperChallenge.
 - Submitted by Sunday evening or Monday, never the morning of the deadline.
-

Owo Reach is payroll for the informal economy: one list in, everyone paid, bank account or not, with every naira accounted for, built on the one payment rail in Nigeria that reaches all the way to cash.